Deteksi Foto dan Perhitungan Uang Rupiah Menggunakan AI, Berbasis Web

KiraUangKu



OLEH :
SECILIA TARIGAN - 18122012
SULTHAN MIFTAHUL U - 18122015

TABLE OF CONTENT

01 PENDAHULUAN

02 GAMBARAN UMUM SISTEM DAN FUNGSIONALITASNYA

03 METODOLOGI DAN IMPLEMENTASI

04 HASIL

PENDAHULUAN

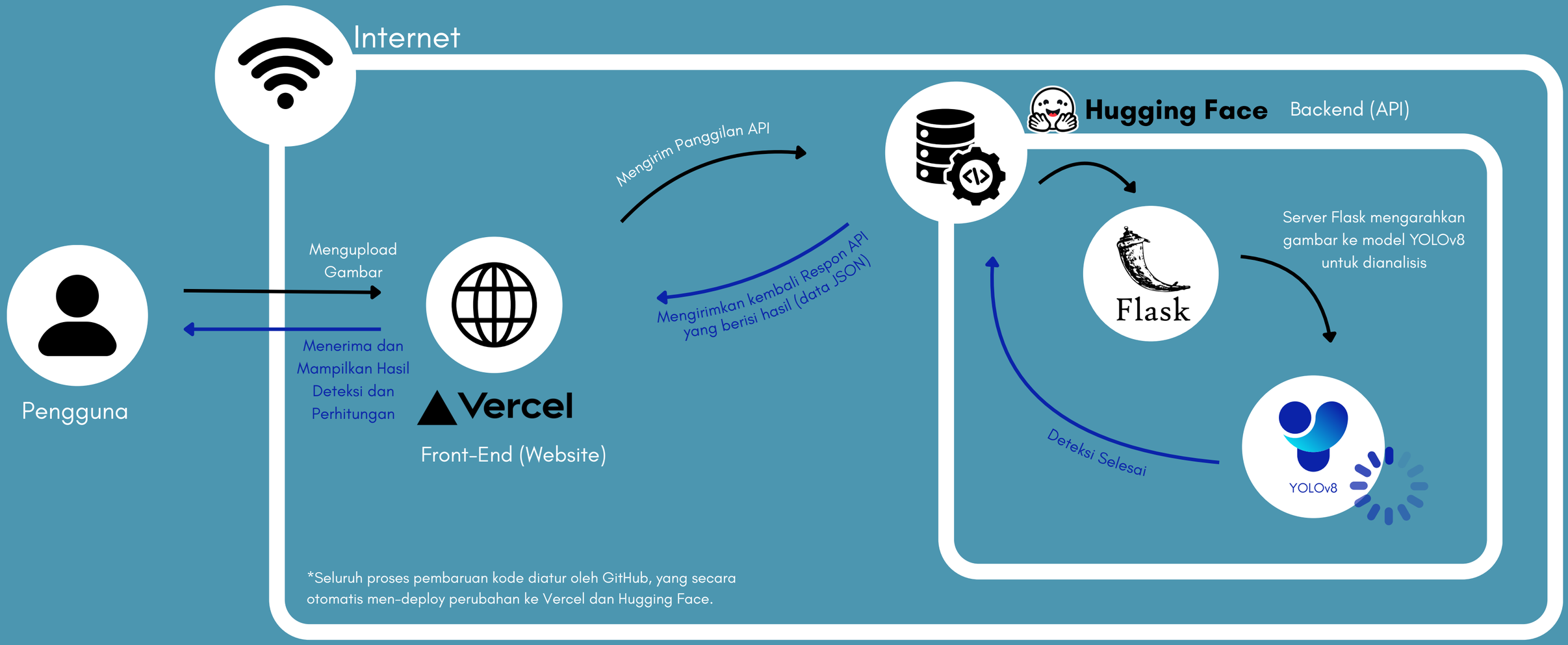
Latar Belakang :

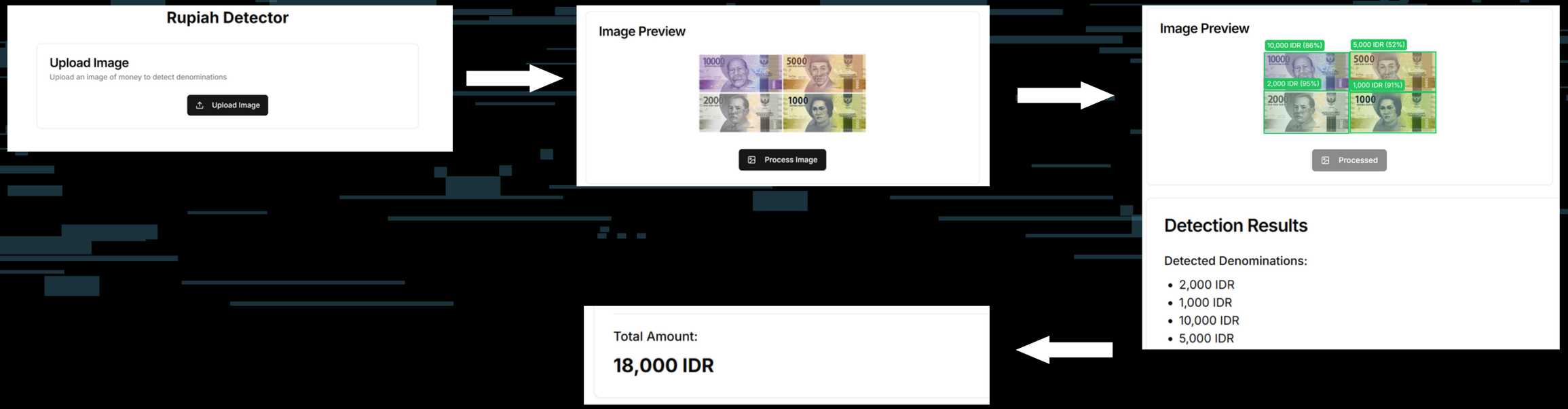
- Proyek ini merupakan tugas besar mata kuliah Layanan Tersambung dan Komputasi Awan.
- Menggabungkan konsep Machine Learning yang telah dipelajari di semester lalu.
- Menjadi kelanjutan praktis dengan memanfaatkan teknologi cloud untuk membangun aplikasi berbasis AI.
- Kami membuat aplikasi web yang dapat mendeteksi dan menghitung nominal uang dari gambar.

Tujuan :

- Otomatisasi proses penghitungan uang melalui gambar.
- Menyediakan pengalaman belajar praktis dalam membangun dan menerapkan model AI.
- Memahami proses end-to-end: data preprocessing, pelabelan, pelatihan model, deployment, dan integrasi ke web app.

GAMBARAN UMUM SISTEM





FUNGSIONALITAS APLIKASI

- MENGUNGGAH GAMBAR
- PREVIEW GAMBAR
- HASIL DETEKSI
- TOTAL JUMLAH UANG

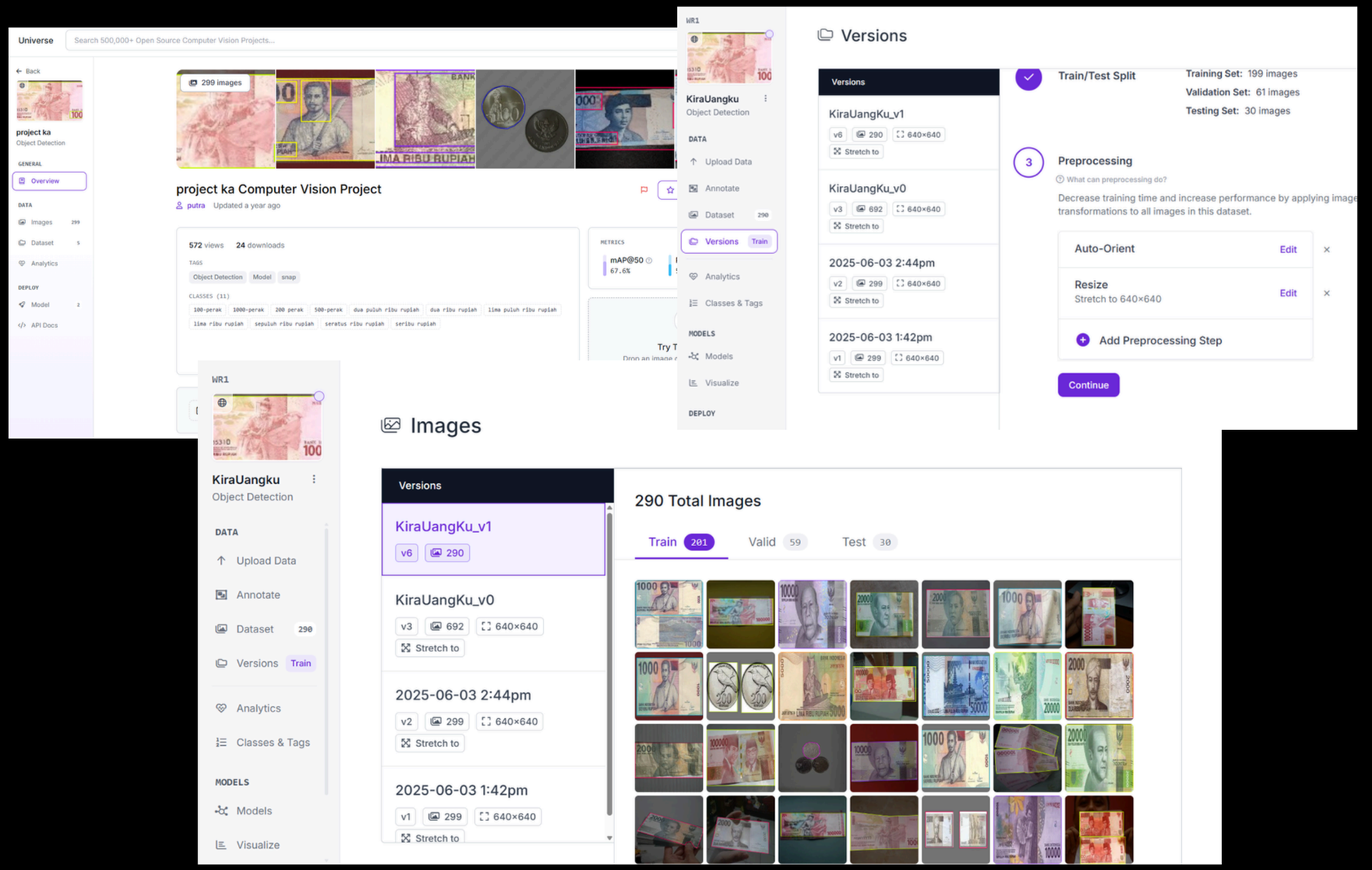
METODOLOGI DAN IMPLEMENTASI

- PENGUMPULAN DAN PRA-PEMROSESAN DATA
- PELABELAN DATA
- PELATIHAN MODEL AI
- PENGEMBANGAN APLIKASI BACKEND (API)
- PENGEMBANGAN APLIKASI WEB FRONT END
- INTEGRASI FRONT END & BACKEND

METODOLOGI DAN IMPLEMENTASI

PENGUMPULAN DAN PRA-PEMROSESAN DATA

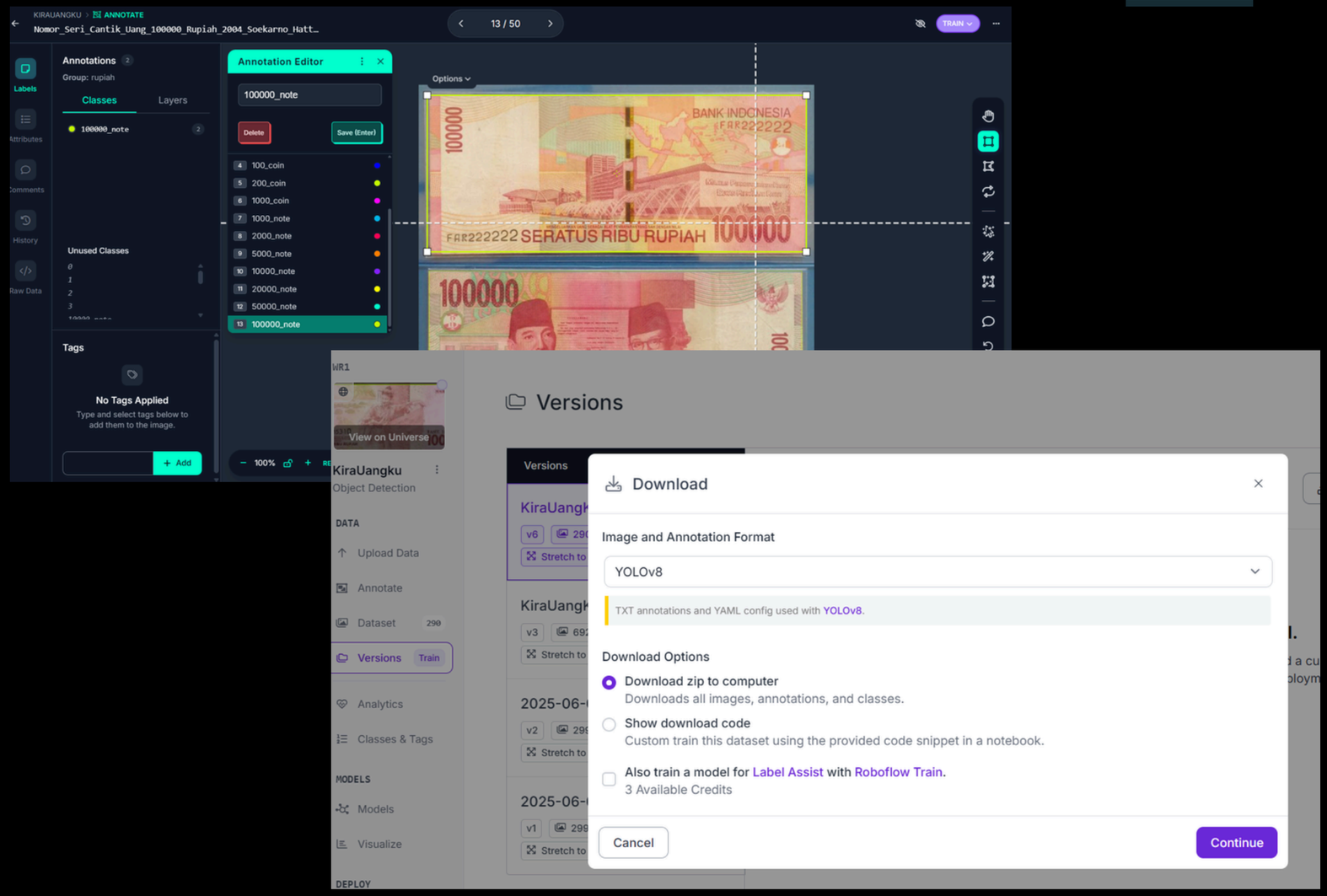
- Sumber Dataset: Platform Roboflow.
- Ukuran & Jenis Data: 299 foto ➡ 290 foto dataset yang digunakan.
- Proses Pra-pemrosesan:
  - Auto-orient
  - Resize: 640x640 piksel untuk menyesuaikan kebutuhan pelatihan model YOLOv8.



METODOLOGI DAN IMPLEMENTASI

PELABELAN DATA

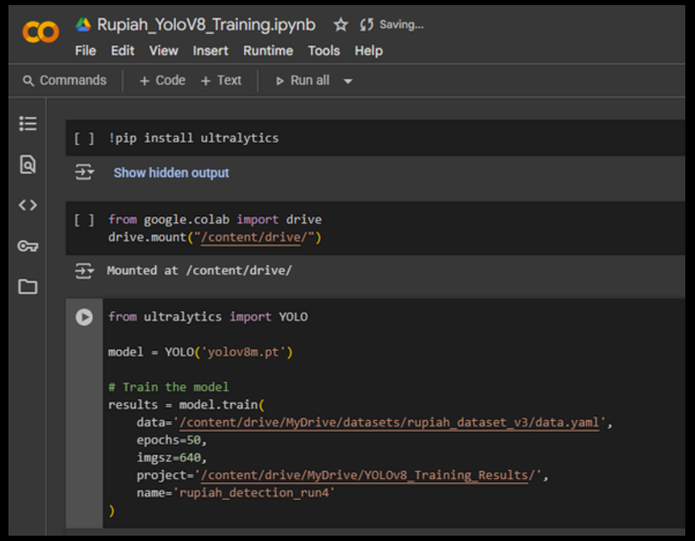
- Tool Pelabelan: Pelabelan (annotasi) data dilakukan menggunakan Roboflow.
- Format Output: Hasil pelabelan diekspor dalam format TXT annotation dan YAML config, yang kompatibel dengan pelatihan model YOLOv8.



METODOLOGI DAN IMPLEMENTASI

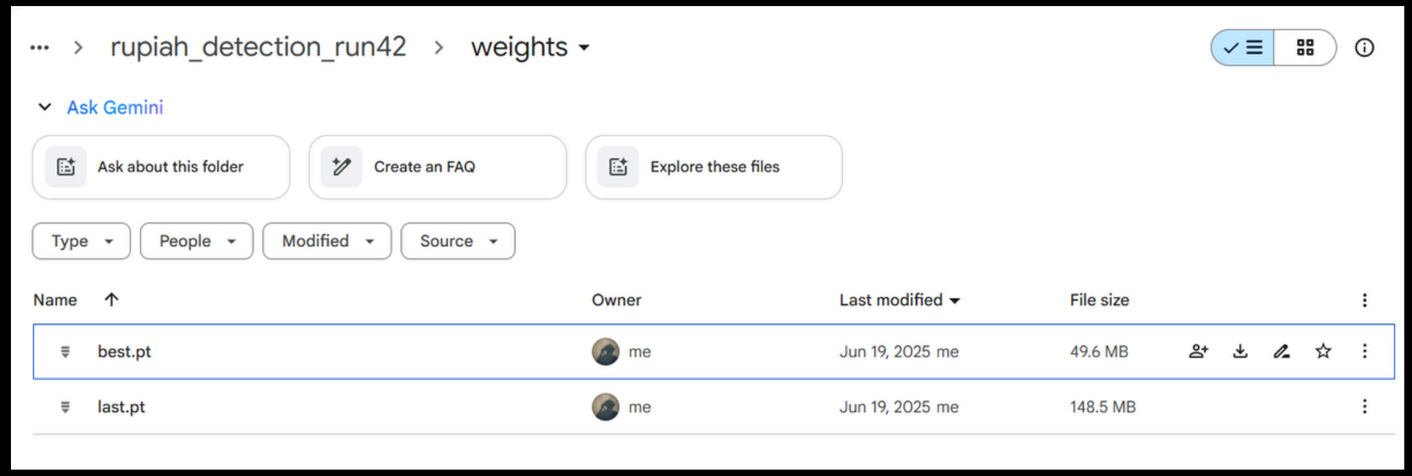
PELATIHAN MODEL AI

- Metode: YOLOv8 (You Only Look Once versi 8)
- Lingkungan Pelatihan: Google Colab
- Hasil Pelatihan: File best.pt




```
50 epochs completed in 0.150 hours.
Optimizer stripped from /content/drive/MyDrive/YOLOv8_Training_Results/rupiah_detection_run42/weights/last.pt, 52.0MB
Optimizer stripped from /content/drive/MyDrive/YOLOv8_Training_Results/rupiah_detection_run42/weights/best.pt, 52.0MB

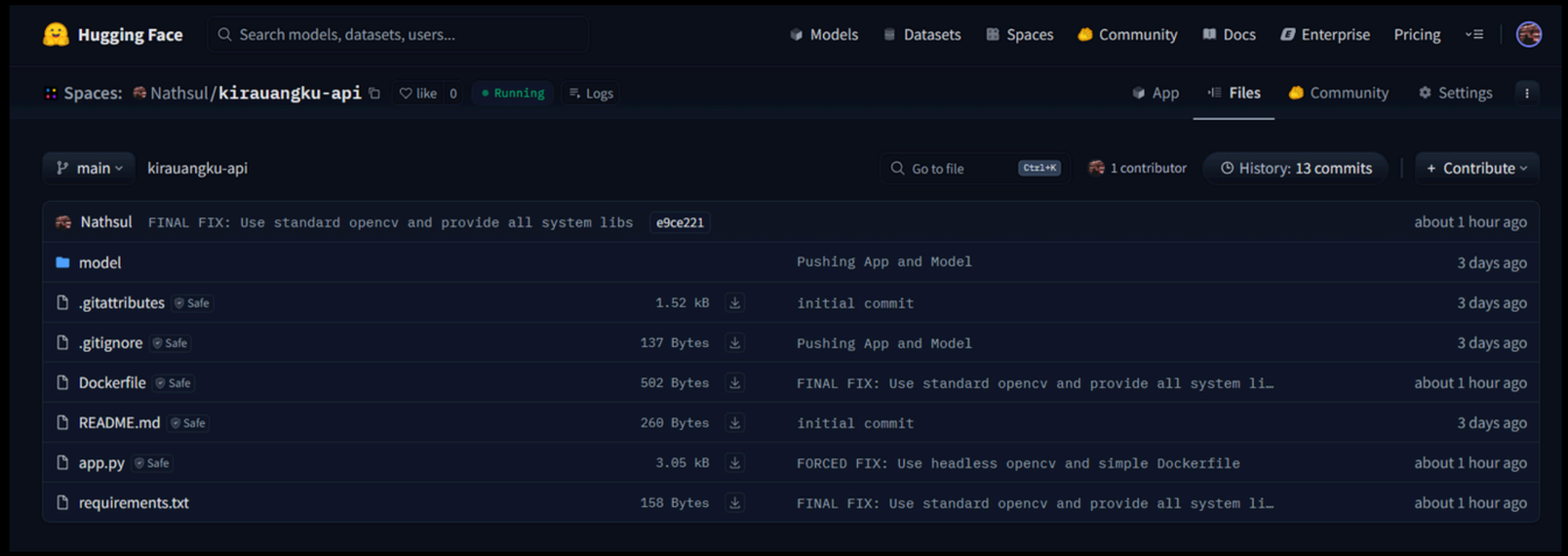
Validating /content/drive/MyDrive/YOLOv8_Training_Results/rupiah_detection_run42/weights/best.pt...
Ultralytics 8.3.156 🚀 Python-3.11.13 torch-2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)
Model summary (fused): 92 layers, 25,845,550 parameters, 0 gradients, 78.7 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 2/2 [00:01<00:00,  1.65it/s]
                   all         59         69      0.818      0.903      0.924      0.786
          100000_note          8          9      0.875          1      0.995      0.792
           10000_note          5          6      0.788          1      0.995      0.974
            1000_coin          1          1      0.584          1      0.995      0.895
            1000_note          6          7        0.8          1      0.995       0.75
             100_coin          4          4          1       0.44      0.828      0.381
           20000_note          7          9      0.852          1      0.984      0.924
            2000_note          8         12      0.877      0.592      0.791      0.728
             200_coin          2          2      0.544          1      0.663      0.622
           50000_note          9         10      0.937          1      0.995      0.869
            5000_note          9          9      0.927          1      0.995      0.922
Speed: 0.3ms preprocess, 12.2ms inference, 0.0ms loss, 2.0ms postprocess per image
```

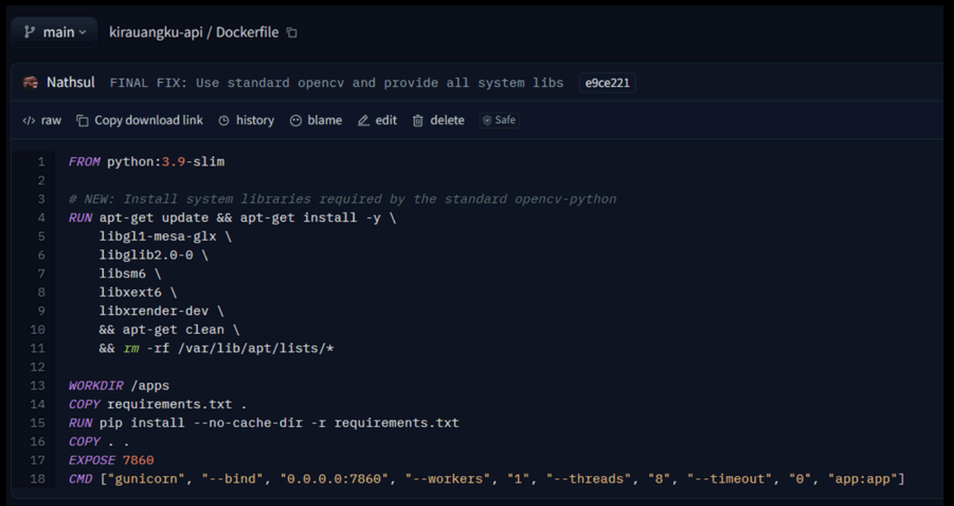


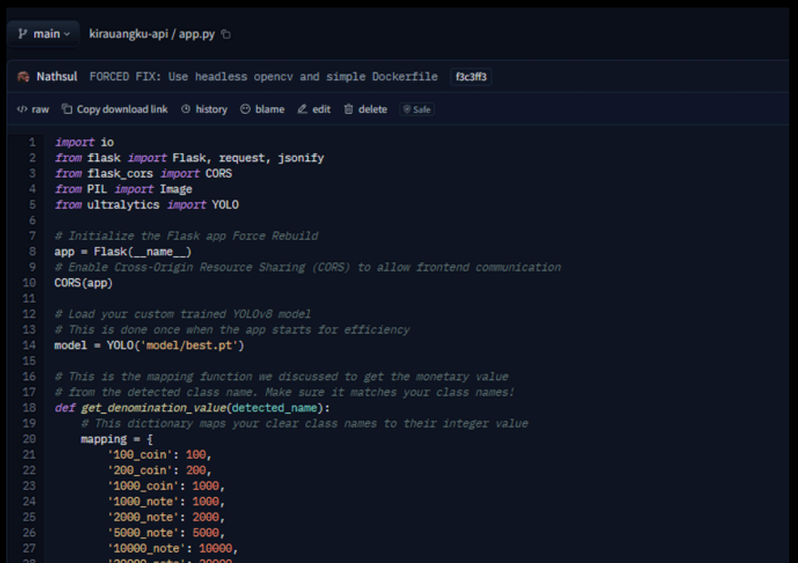
METODOLOGI DAN IMPLEMENTASI

PENGEMBANGAN APLIKASI BACKEND (API)

- Pembuatan Inti API Menggunakan Flask
- Kontainerisasi Aplikasi dengan Docker
- Deployment Otomatis ke Cloud (Hugging Face Spaces)







PENGEMBANGAN APLIKASI WEB: FRONT-END

Framework: Next.js.

Fitur-fitur Utama:

- Upload Section
- Preview Section
- Results Display

```
17  export default function MoneyDetector() {
52    const processImage = async () => {
63      try {
64        const formData = new FormData()
65        formData.append("file", file)
66
67        const apiUrl = "https://nathsul-kirauangku-api.hf.space/detect"
68
69        const response = await fetch(apiUrl, {
70          method: "POST",
71          body: formData,
72        })
```

INTEGRASI FRONT-END & BACK-END (API)

Komunikasi API: Front-end berkomunikasi dengan backend melalui permintaan deteksi gambar dikirimkan ke endpoint API spesifik /detect pada backend.

Proses Data: Gambar diunggah dari front-end ke backend, backend memprosesnya menggunakan model AI, dan mengirimkan kembali hasil deteksi (termasuk total jumlah) dalam format JSON ke front-end untuk ditampilkan.

METODOLOGI DAN IMPLEMENTASI

PROMPTING AI FRONT-END

Create a clean and minimal web page using React and Tailwind CSS that allows a user to upload an image of money. After the image is uploaded, show a preview of the image and a "Process" button. When the "Process" button is clicked, simulate an API call to an object detection model and display the detected denominations (like "50,000", "20,000", etc.) with bounding boxes over the image. Then, calculate and display the total amount of money detected below the image. Use placeholders for the API response and show mock predictions for testing (e.g., "Detected: 50,000 IDR, 20,000 IDR, Total: 70,000 IDR"). The layout should be responsive, center-aligned, and mobile-friendly. Use cards or sections to separate upload, preview, and result areas.

HASIL

KIRA-UANGKU.VERCEL.APP

Rupiah Detector

Upload Image

Upload an image of money to detect denominations

Upload Image

Rupiah Detector

Upload Image

Upload an image of money to detect denominations

Upload Image

Image Preview



Process Image



Processed

Detection Results

Detected Denominations:

- 5,000 IDR
- 10,000 IDR
- 1,000 IDR
- 2,000 IDR

Total Amount:

18,000 IDR

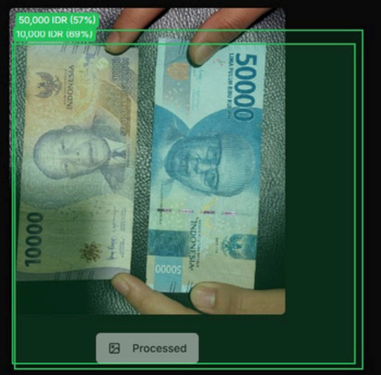
Rupiah Detector

Upload Image

Upload an image of money to detect denominations

Upload Image

Image Preview



Detection Results

Detected Denominations:

- 10,000 IDR
- 50,000 IDR

Total Amount:

60,000 IDR

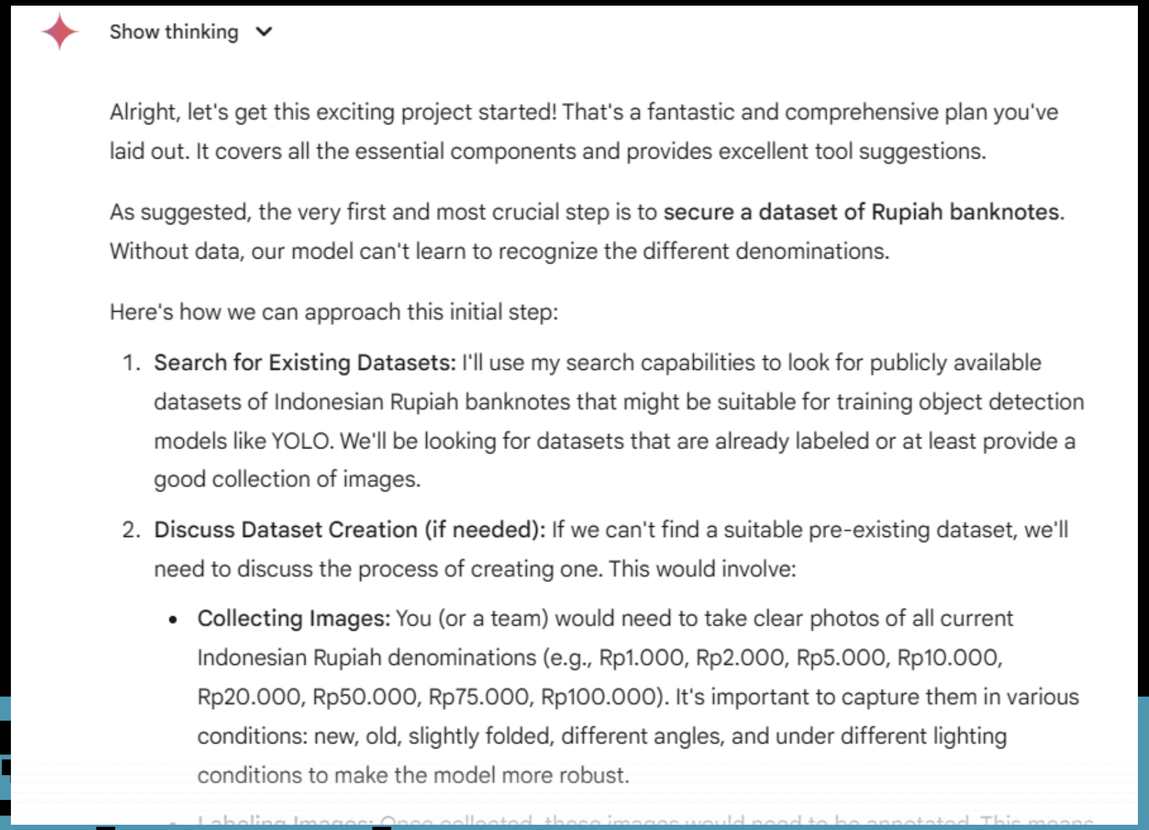
THANK YOU

ANY QUESTIONS?

PROMPT GEMINI

Kami menggunakan ai untuk 2 keperluan:

1. Outline Pengerjaan Tugas



Show thinking

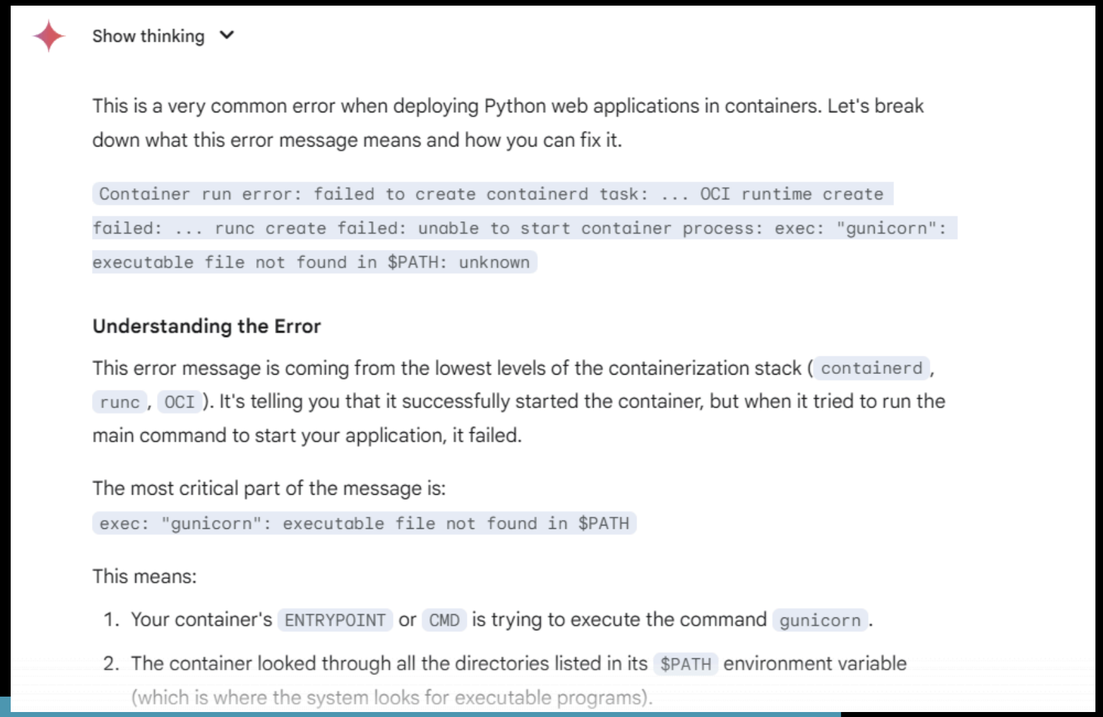
Alright, let's get this exciting project started! That's a fantastic and comprehensive plan you've laid out. It covers all the essential components and provides excellent tool suggestions.

As suggested, the very first and most crucial step is to **secure a dataset of Rupiah banknotes**. Without data, our model can't learn to recognize the different denominations.

Here's how we can approach this initial step:

1. **Search for Existing Datasets:** I'll use my search capabilities to look for publicly available datasets of Indonesian Rupiah banknotes that might be suitable for training object detection models like YOLO. We'll be looking for datasets that are already labeled or at least provide a good collection of images.
2. **Discuss Dataset Creation (if needed):** If we can't find a suitable pre-existing dataset, we'll need to discuss the process of creating one. This would involve:
   - **Collecting Images:** You (or a team) would need to take clear photos of all current Indonesian Rupiah denominations (e.g., Rp1.000, Rp2.000, Rp5.000, Rp10.000, Rp20.000, Rp50.000, Rp75.000, Rp100.000). It's important to capture them in various conditions: new, old, slightly folded, different angles, and under different lighting conditions to make the model more robust.
   - **Labeling Images:** Once collected, these images would need to be annotated. This means

2. Troubleshoot Coding



Show thinking

This is a very common error when deploying Python web applications in containers. Let's break down what this error message means and how you can fix it.

`Container run error: failed to create containerd task: ... OCI runtime create failed: ... runc create failed: unable to start container process: exec: "gunicorn": executable file not found in $PATH: unknown`

Understanding the Error

This error message is coming from the lowest levels of the containerization stack (`containerd` , `runc` , `OCI`). It's telling you that it successfully started the container, but when it tried to run the main command to start your application, it failed.

The most critical part of the message is:
`exec: "gunicorn": executable file not found in $PATH`

This means:

1. Your container's `ENTRYPOINT` or `CMD` is trying to execute the command `gunicorn` .
2. The container looked through all the directories listed in its `$PATH` environment variable (which is where the system looks for executable programs).